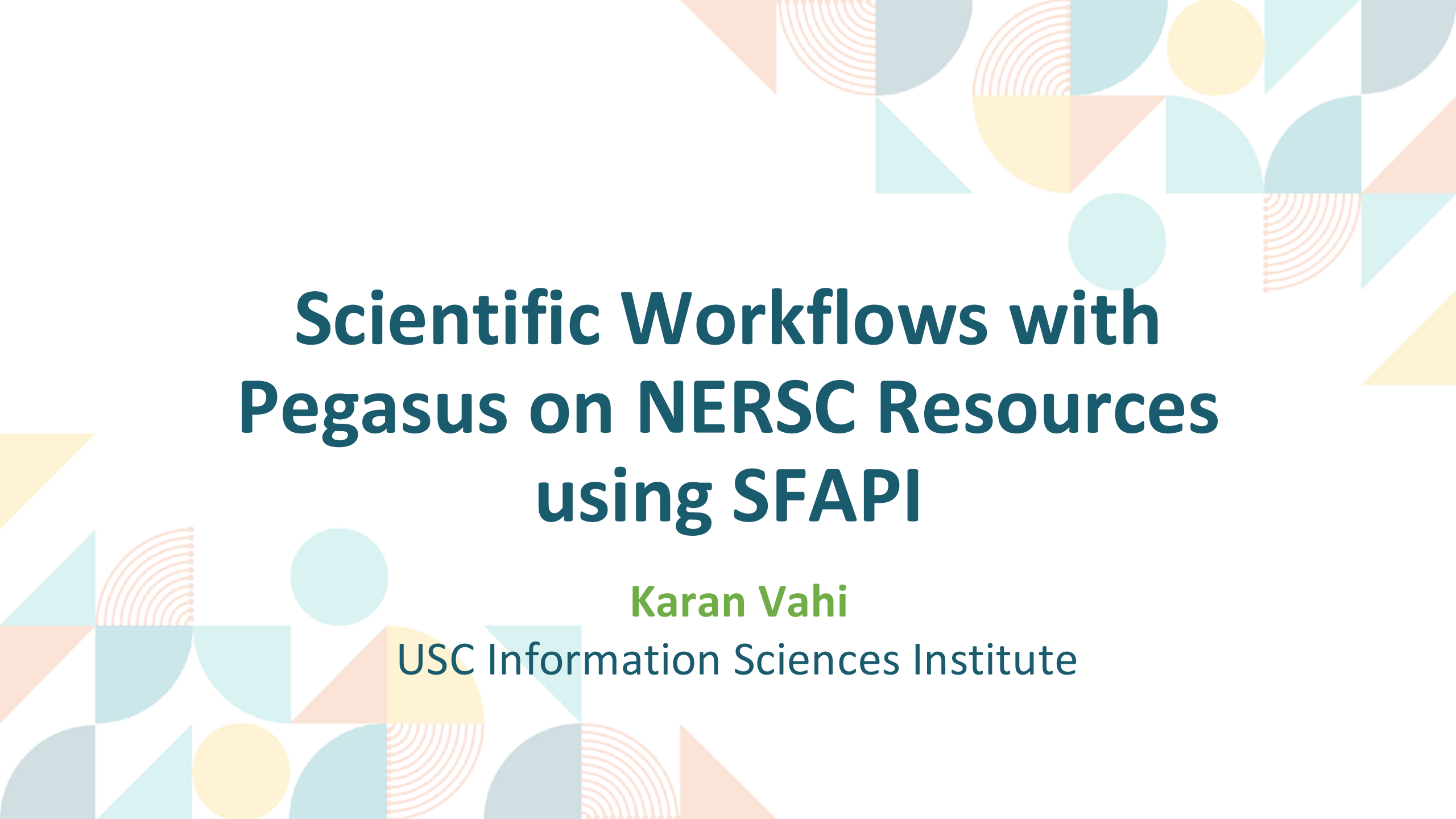




Scientific Workflows with Pegasus on NERSC Resources using SFAPI

Karan Vahi

USC Information Sciences Institute

The Challenge: Moving Beyond the Laptop

The Scripting Trap



Quick to code, but hard to scale

Difficult to maintain and error-prone

Hard to move between resources (Laptop vs. Cluster)

“Works on my machine” syndrome

The Workflow Advantage



Declarative: Describe what to do, not how

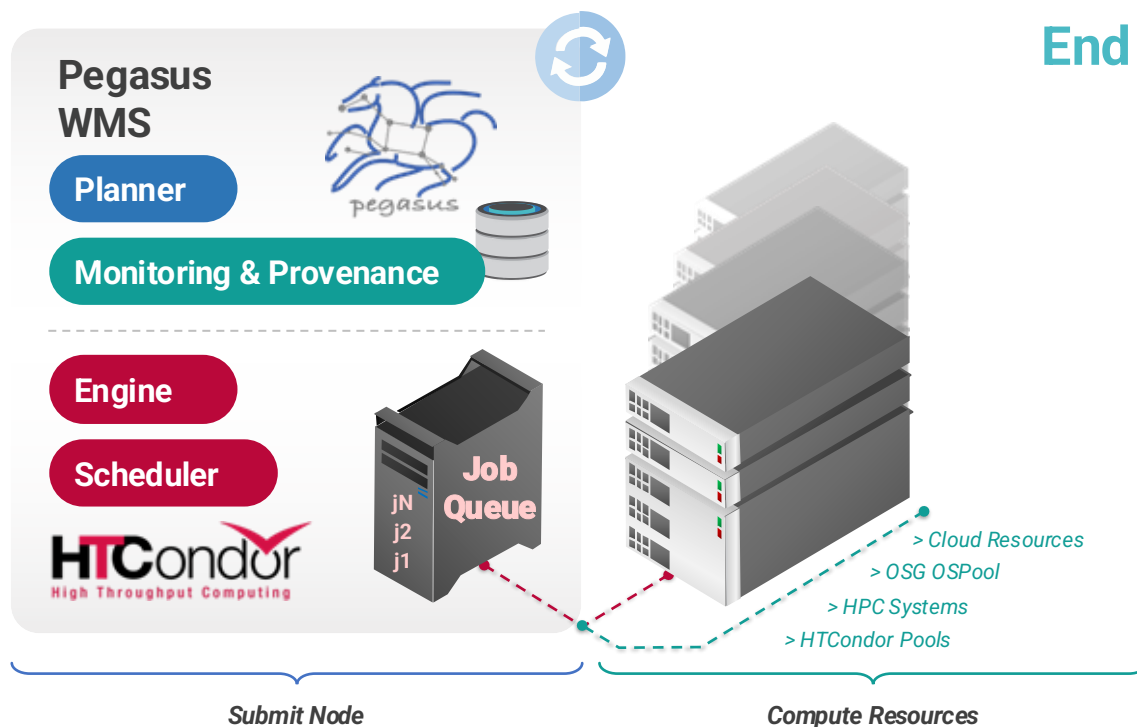
Automated: Minimizes human involvement

Provenance: Tracks history for reproducibility

Portable: Runs on ACCESS, NAIRR, cloud, ...



Pegasus Workflow Management System

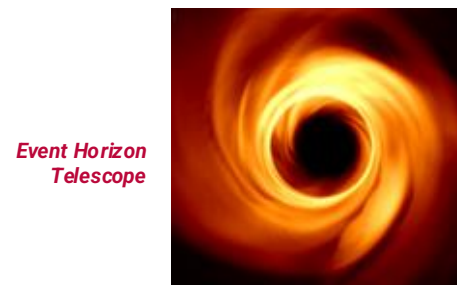


End to End Workflow Management & Execution

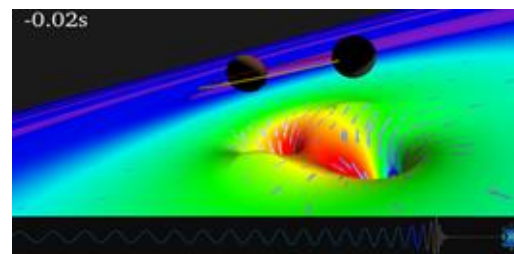
- ▶ Develop portable scientific workflows in Python and Java
- ▶ Compile workflows to be run on heterogeneous resources
- ▶ Monitor and debug workflow execution via CLI and web-based tools
- ▶ Recover from failures with built-in fault tolerance mechanisms
- ▶ Regular release schedule incorporating latest research and development



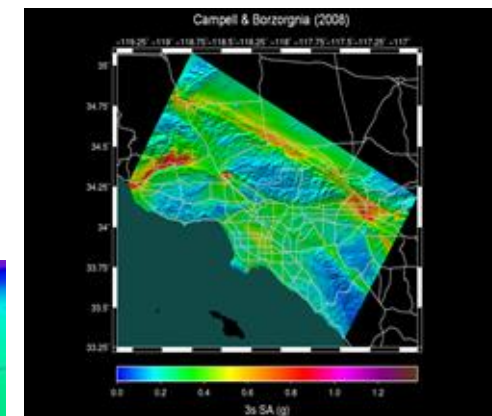
- ▶ Laser Interferometer Gravitational Wave Observatory (LIGO) develops large scale analysis pipelines used for gravitational wave detection.
- ▶ Southern California Earthquake Center (SCEC) CyberShake project generates hazard maps using hierarchical workflows.
- ▶ The Event Horizon Telescope uses Pegasus to orchestrate and scale complex distributed computing pipelines for data calibration, massive synthetic data generation, and deep learning inference



Event Horizon Telescope



LIGO observation of colliding black holes



Hazard map indicating maximum amount of shaking at a particular geographic location generated from SCEC's CyberShake Pegasus workflow

Application Showcase: <https://pegasus.isi.edu/application-showcase/>

Key Pegasus Concepts

▲ Pegasus WMS == Pegasus planner (mapper) + DAGMan workflow engine + HTCondor scheduler/broker

- Pegasus maps workflows to infrastructure
- DAGMan manages dependencies and reliability
- HTCondor is used as a broker to interface with different schedulers

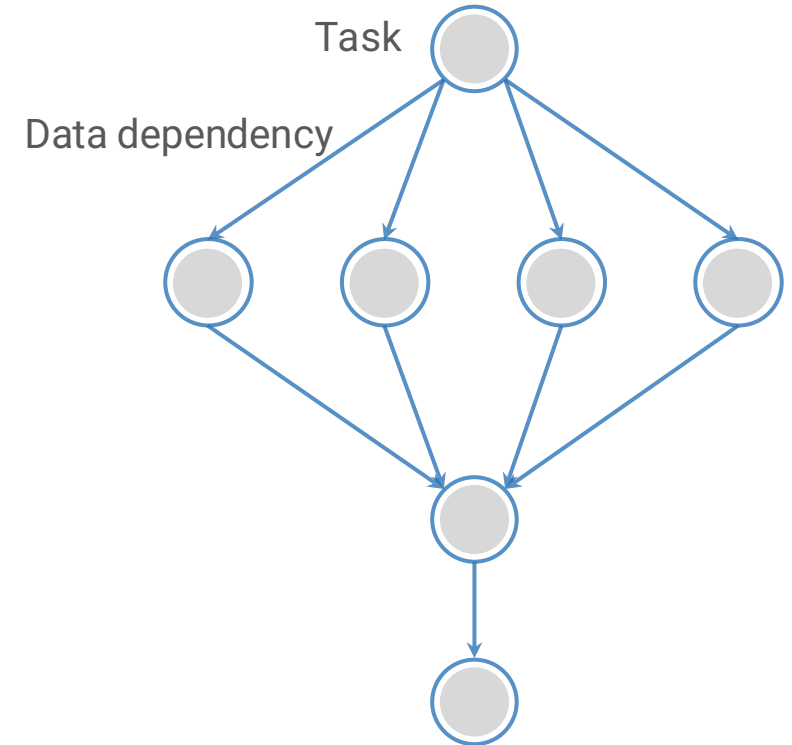
▲ Workflows are DAGs

- Nodes: jobs, edges: dependencies
- No while loops, no conditional branches
- Jobs are standalone executables

▲ Planning occurs ahead of execution

▲ Planning converts an abstract workflow into a concrete, executable workflow

- Planner is like a compiler



Defining Workflows in Python

```
from Pegasus.api import *

# Create abstract workflow
wf = Workflow("pipeline")

# Track the data
page = File("page.html")

# Create a Job
curl_job = Job("curl") \
    .add_args("-o", page, "https://pegasus.isi.edu") \
    .add_outputs(page)

# Create a Dependent Job
wc_job = Job("wc") \
    .add_args("-l", page) \
    .add_inputs(page)

# Add jobs and dependencies
wf.add_jobs(curl_job, wc_job)
wf.add_dependency(wc_job, parents=[curl_job])
```

Declarative job addition

**Simple dependency
definition**

**Generates the abstract
workflow (DAG)**

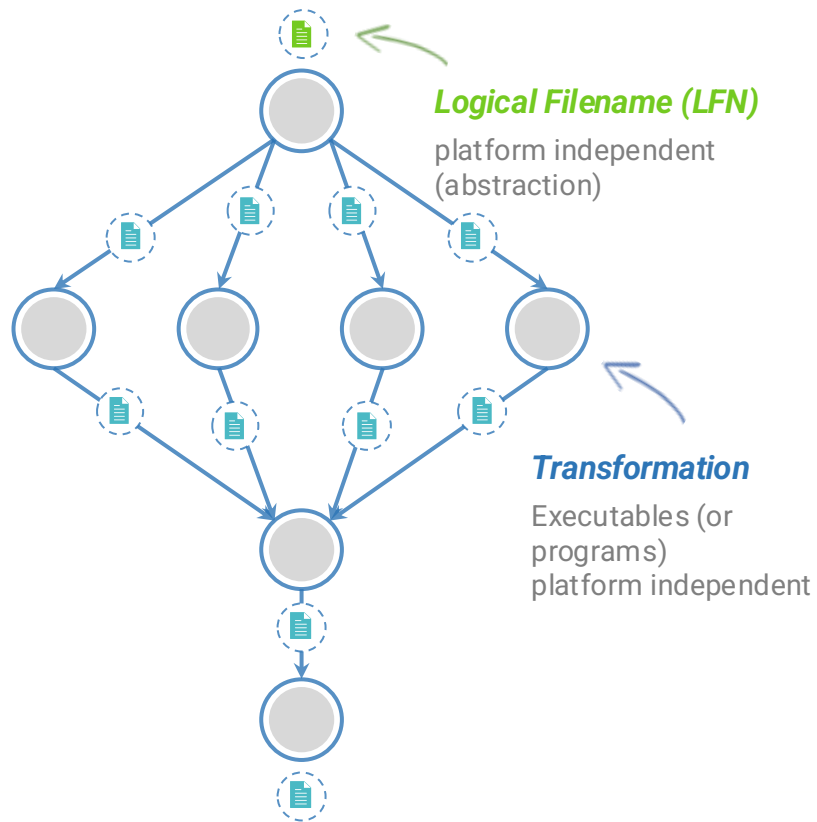
Input Workflow Specification

YAML
formatted

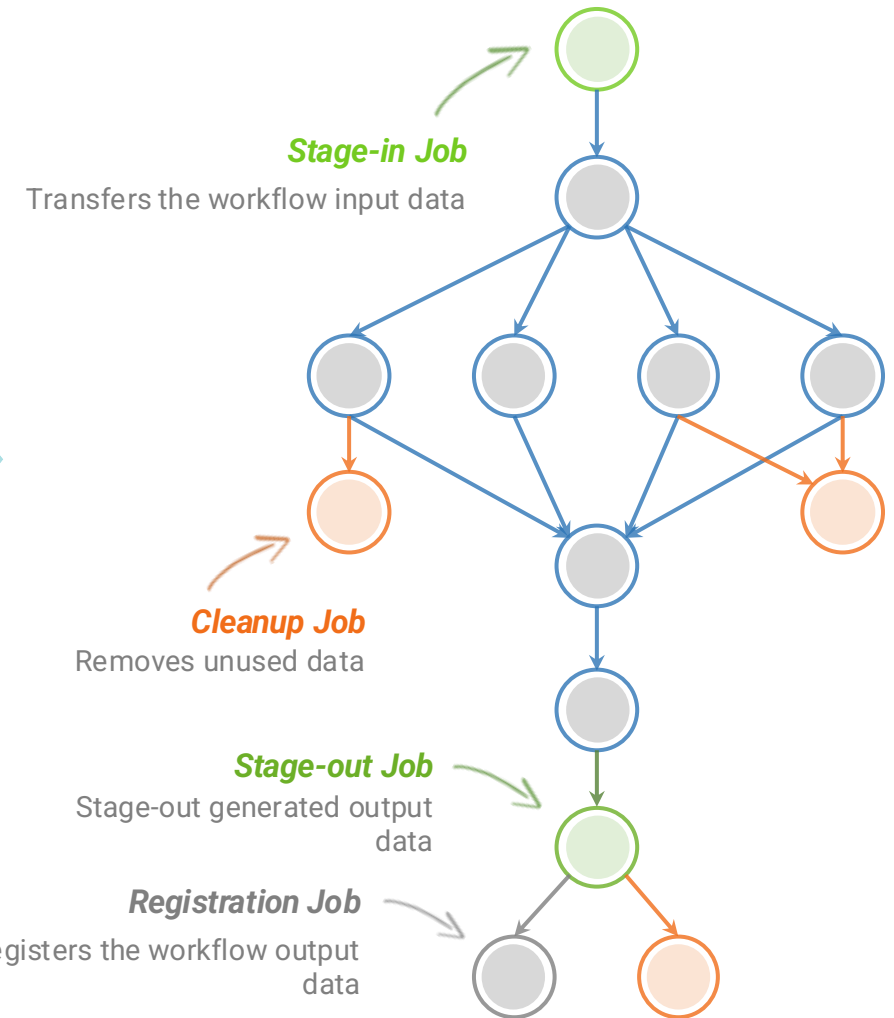
Portable Description

Users do not worry about low level execution details

ABSTRACT WORKFLOW

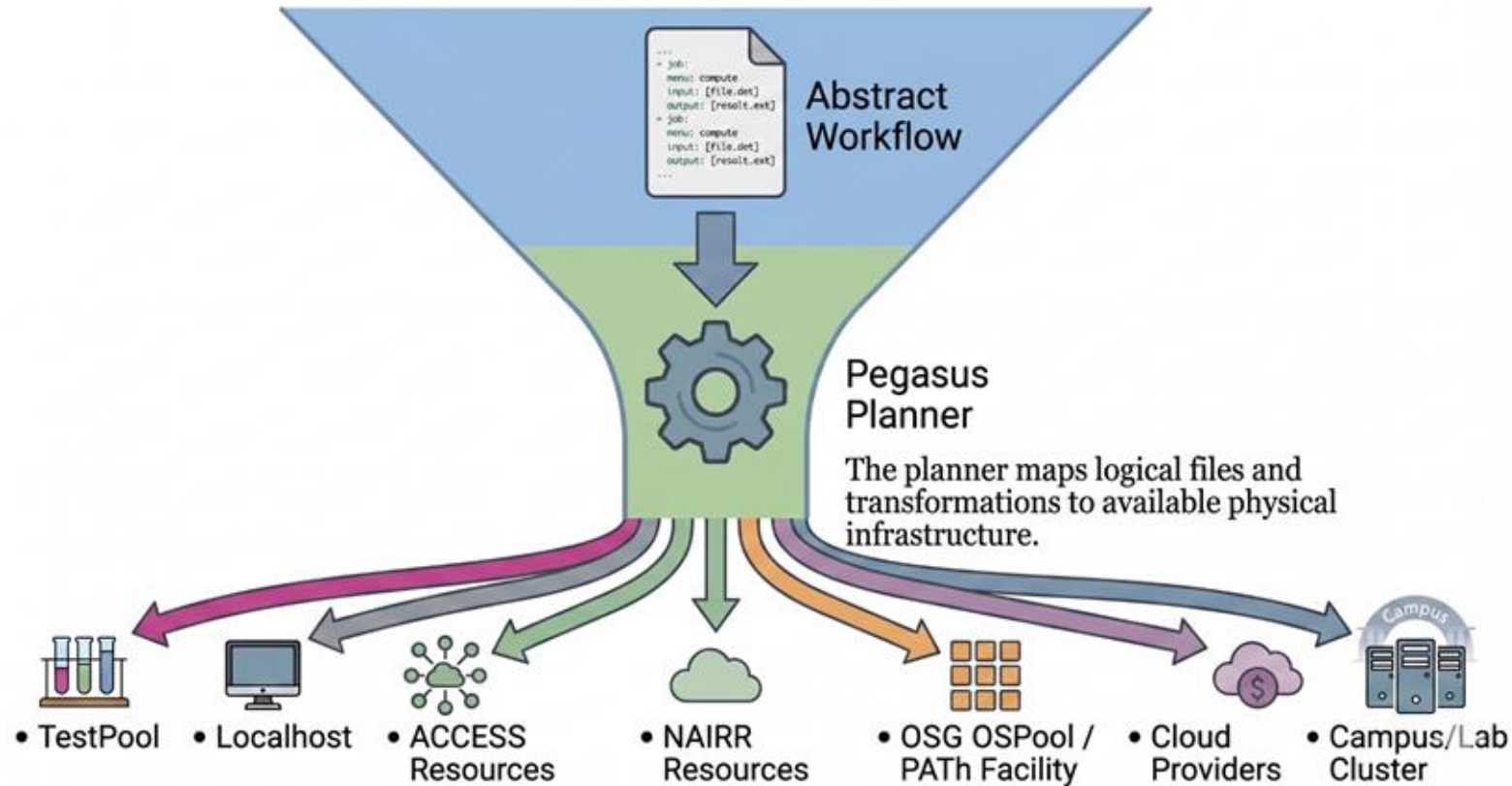


Output Workflow



EXECUTABLE WORKFLOW

Write Once, Run Anywhere



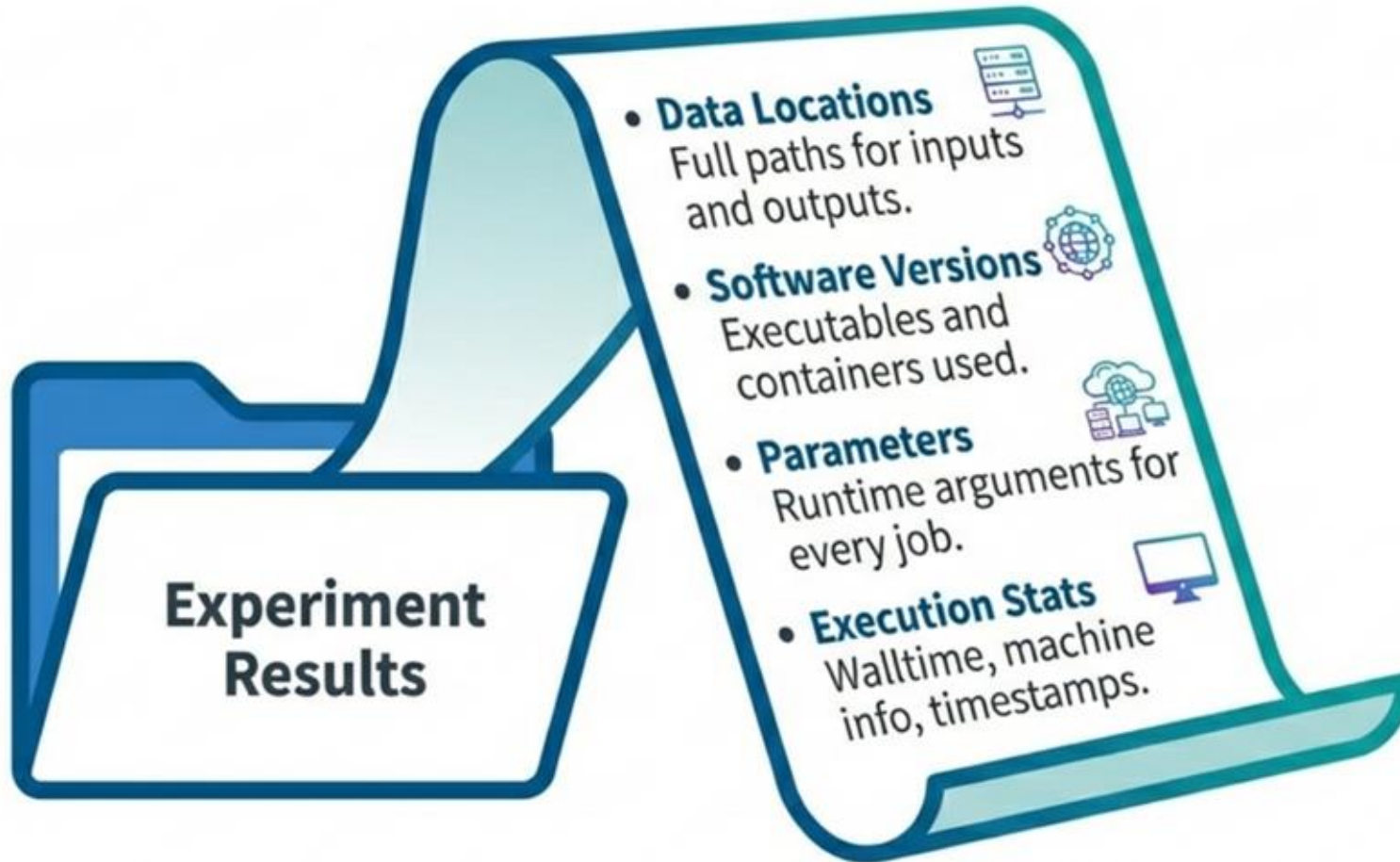
Requirement: Any HPC/cloud/... system with outbound network connectivity.
Includes your own campus cluster or lab machines!

Resilience: When Things Go Wrong



Provenance & Reproducibility

Good science requires



Pegasus tracks every bit of **data** and **every compute** step, ensuring your results are **verifiable** and **reproducible**.

The NERSC Superfacility API

NERSC's **Superfacility API (SFAPI)** is a REST API for scripting jobs and workflows against NERSC systems.

What you can do through the API



System status

Check if systems are up; no client needed



Submit jobs

Launch batch jobs from your own scripts



Track jobs

Poll job state as a workflow runs



Move files

Transfer data in and out of NERSC



Reservations

Book compute time in advance



Accounting

Query usage for your projects

How access works

- 1 Create a client in Iris**
Its security level sets lifetime and allowed source IPs
- 2 Get an access token**
Sign with the client key; tokens last about 10 minutes
- 3 Call the API**
At api.nersc.gov/api/v1.2 or with the `sfapi_client` Python library

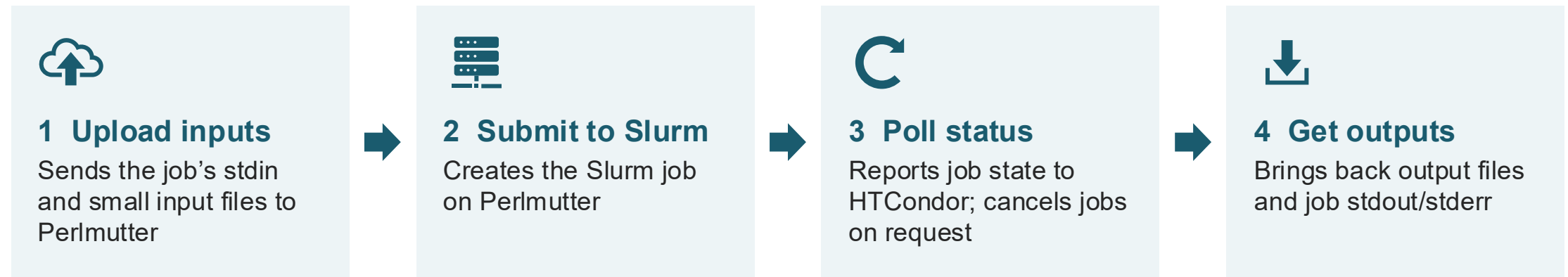
Source: NERSC Documentation, "The Superfacility API" docs.nersc.gov/services/sfapi



How Pegasus Uses the Superfacility API

Pegasus uses SFAPI now to submit and monitor workflow jobs on Perlmutter from a remote submit host. The integration has been done in the HTCondor BLAHP layer to submit jobs using the *“sfapi_client”* python library.

For every job, HTCondor's SFAPI plugin calls the API over HTTPS to:



No SSH tunnel or MFA prompt

Calls are authenticated with a client key registered in Iris

No shared filesystem

Submit host and Perlmutter need no common storage

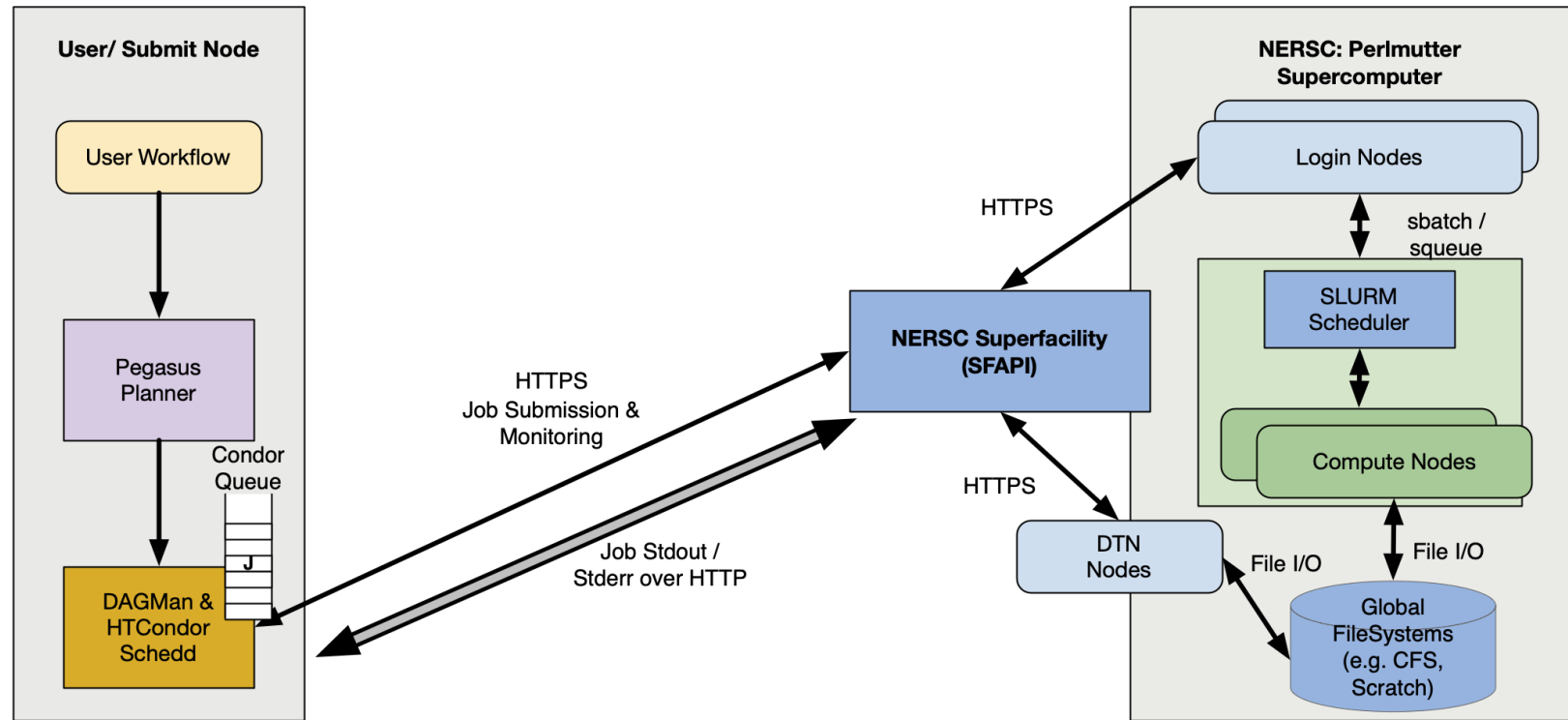
Same workflow code

Only the site catalog entry for Perlmutter changes

Source: Pegasus User Guide <https://pegasus.isi.edu/documentation/user-guide/deployment-scenarios.html#nersc-perlmutter-via-sfapi>



Pegasus WMS Job Submission to NERSC via SFAPI



Setting Up Pegasus for NERSC SFAPI

Setup on the submit host

1 Register an SFAPI client in Iris

Choose the **Red** security level (job submission, 2-day lifetime) and include the submit host's IP. Save the client ID and keys to `~/.superfacility/`

2 Create the Python environment

Python 3.11+ virtualenv at `~/superfacility-env` with `pip install sfapi_client`. Python 3.9 environments fail to submit.

3 Get an SSH key for data transfers

Run `sshproxy` on the submit host to get `~/.ssh/nersc`, valid 24 hours without MFA; set it as the local site's `SSH_PRIVATE_KEY` profile.

Credential errors? `chmod 600` the keys; check client scopes.

Site catalog entry for Perlmutter (key lines)

```
- name: perlmutter
  grids:
    - type: sfapi
      contact: perlmutter
      scheduler: slurm
      jobtype: compute
  pegasus:
    style: glite
    queue: shared
    project: ${RESOURCE_PROJECT}
    data.configuration: nonsharedfs
    glite.arguments: -C cpu
```

Scratch and outputs live on CFS:

`/global/cfs/cdirs/<project>/<user>/`

Latest entry: github.com/pegasushub/pegasus-site-catalogs

Source: Pegasus 5.1 User Guide, “NERSC Perlmutter via SFAPI” (Prerequisites, Site Catalog, Troubleshooting, Data Transfers)



Science Highlight: Hot Springs - What lives in the deep, and what is it doing?

Along the Costa Rican subduction zone, deep hot springs bring fluids up from where one tectonic plate slides beneath another. Microbes live in that heat and chemistry and what they do there helps govern how carbon moves between the Earth's interior and its surface.

The study asks three things

Who is there?

Which microbial communities live in each spring, across the range of temperatures and chemistries.

What can they do?

Which metabolic genes they carry especially the machinery for fixing carbon in the subsurface.

What shapes them?

How the geochemistry of each site drives both the communities and their carbon-cycling potential.

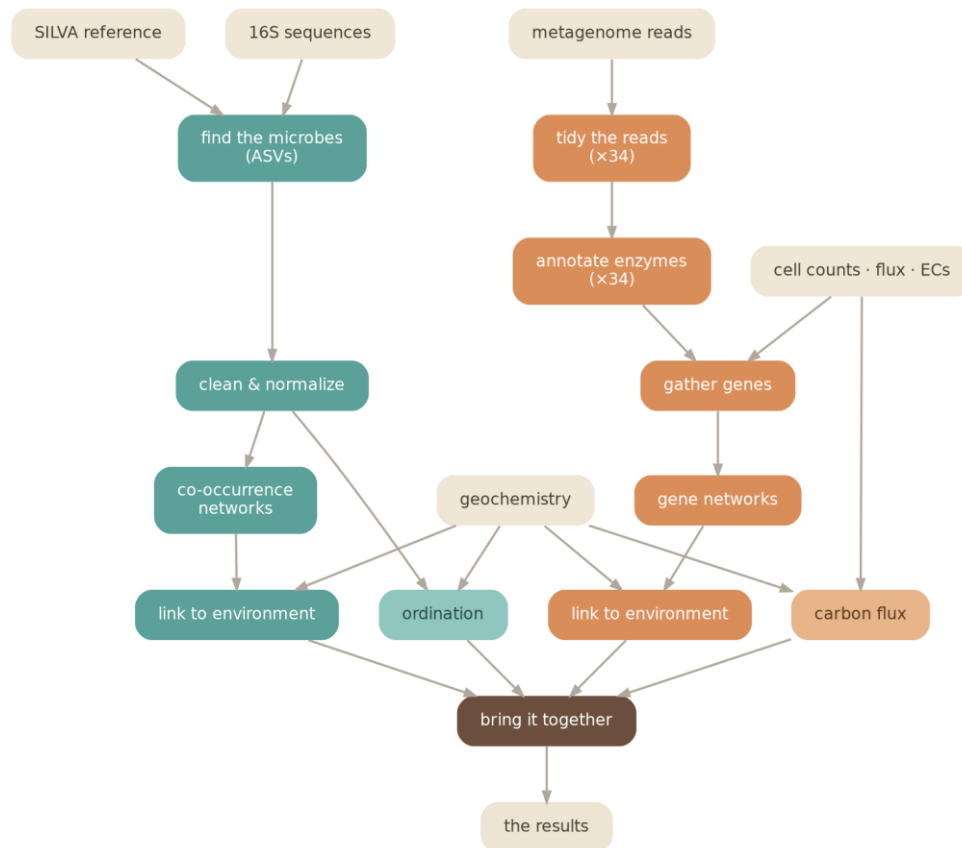
Understanding this deep microbial carbon cycle helps explain how much carbon stays locked in the Earth versus returns to the atmosphere — a piece of the planet's long-term climate balance.

<https://dornsife.usc.edu/lloyd/projects/>



Karen Lloyd – USC Dornsife Hamza Safri – USC/ISI

A two-track pipeline, end to end



The 16S track — the microbes

Processed 16S sequences are grouped by sample, classified against the SILVA reference to name the microbes, cleaned and normalized, then reduced to co-occurrence networks and an ordination — and linked to the site chemistry.

The metagenome track — the chemistry

Raw reads from 34 libraries are quality-trimmed, then annotated with mi-faser to detect metabolic enzymes. Carbon-fixation genes are merged, built into networks, and linked to chemistry — and a carbon-flux estimate is computed per site.

Both tracks converge

The two paths meet in a single report that ties the microbial communities to the chemistry around them — the picture the original study drew, rebuilt from public data.

The full pipeline completed on Perlmutter — all 34 metagenome samples and the 16S community, end to end.



Try Pegasus?

An **easy and convenient** way to try out Pegasus and submit workflows to NERSC is to use **ACCESS Pegasus**.

What ACCESS Pegasus provides

Hosted, web-based

Open OnDemand and Jupyter, with no local install.

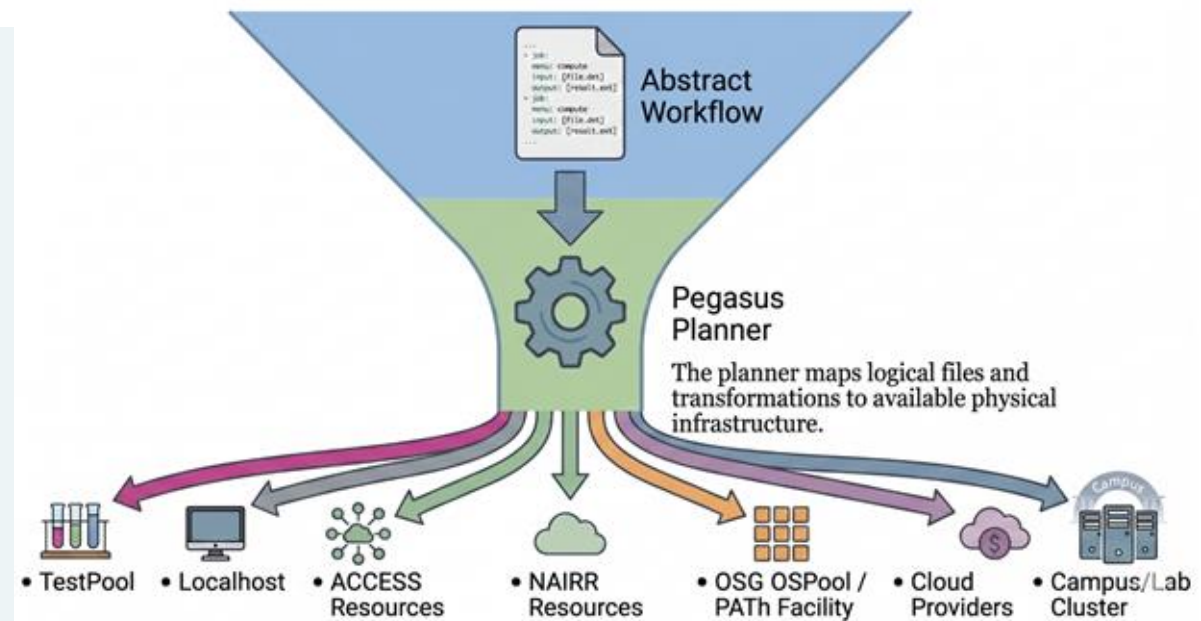
Single sign-on with your ACCESS ID

Any user with an ACCESS id can login. You don't need an ACCESS Allocation.

Training Notebooks

Try out Pegasus training examples

<https://github.com/pegasus-isi/ACCESS-Pegasus-Examples>



Log in: pegasus.access-ci.org

Help: support.access-ci.org/tools/pegasus



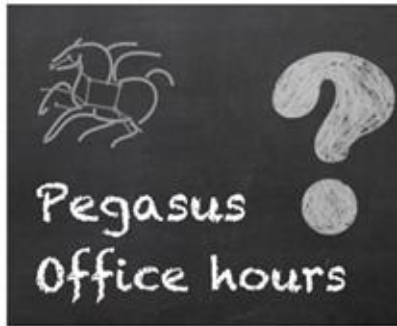
We are here to help!

- Slack channel
- Email: pegasus-support@isi.edu
- Office hours every Friday, 11am PT

Office Hours

Join the Pegasus team every Friday for virtual office hours at 11 AM Pacific / 2 PM Eastern.

Do you have questions about workflows or need guidance on organizing and implementing them? Join our weekly office hours – designed to support both new and experienced users in learning and engaging with Pegasus. Here's what to expect:



- Tutorial walkthrough First Friday of the month
- <https://pegasus.isi.edu>

The header of the Pegasus website, featuring the Pegasus logo (a blue line drawing of a Pegasus head) and the word "Pegasus" in a blue, sans-serif font. To the right are links for "About", "Blog", "AI / ML", and "Learn". Below the header is a large, colorful, abstract graphic consisting of many thin, curved lines in shades of blue, orange, and red, resembling a stylized Pegasus head or a network structure.

[in Share](#) [f Share](#) [X Post](#) [Reddit](#) [Like 1](#)

Slack

[Add to Slack](#) We encourage you to join the Slack Workspace as it is an on-going, open forum for all Pegasus users to share ideas, experiences, and talk out issues with the Pegasus Development team. Please click the button above or ask for an invite by trying to join pegasus-users.slack.com in the Slack app, or send an email to pegasus-support@isi.edu to request an invite.

Mailing Lists

pegasus-users@isi.edu

This list is the main support vehicle for Pegasus. Please note that the following subscription link requires a Google-linked email address. If you want to subscribe with non-linked email address, please contact us at pegasus-support@isi.edu and we will add you.

- [Subscribe](#) / [unsubscribe](#) / [archive](#)

pegasus-announce@isi.edu

Messages about new releases and updates. Low traffic. Please note that the following subscription link requires a Google-linked email address. If you want to subscribe with non-linked email address, please contact us at pegasus-support@isi.edu and we will add you.

- [Subscribe](#) / [unsubscribe](#) / [archive](#)





Thank you!

Questions?

Karan Vahi | vahi@isi.edu

Acknowledgments

For support on the Pegasus SFAPI integration

JF

Jaime Frey

HTCondor team, CHTC, UW–Madison

NT

Nick Tyler

NERSC, Berkeley Lab